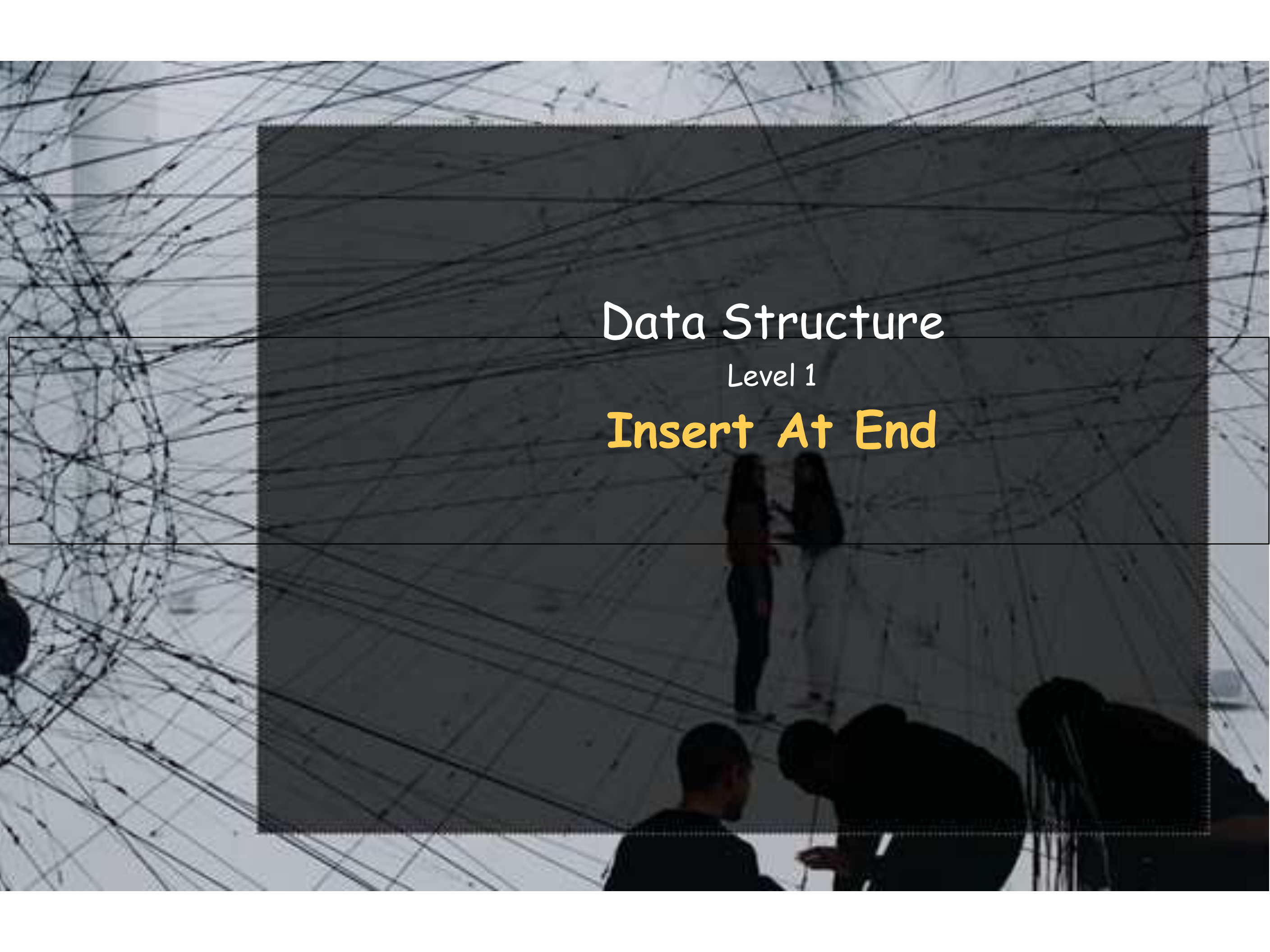


Data Structure

Level 1

Doubly Linked List Operation

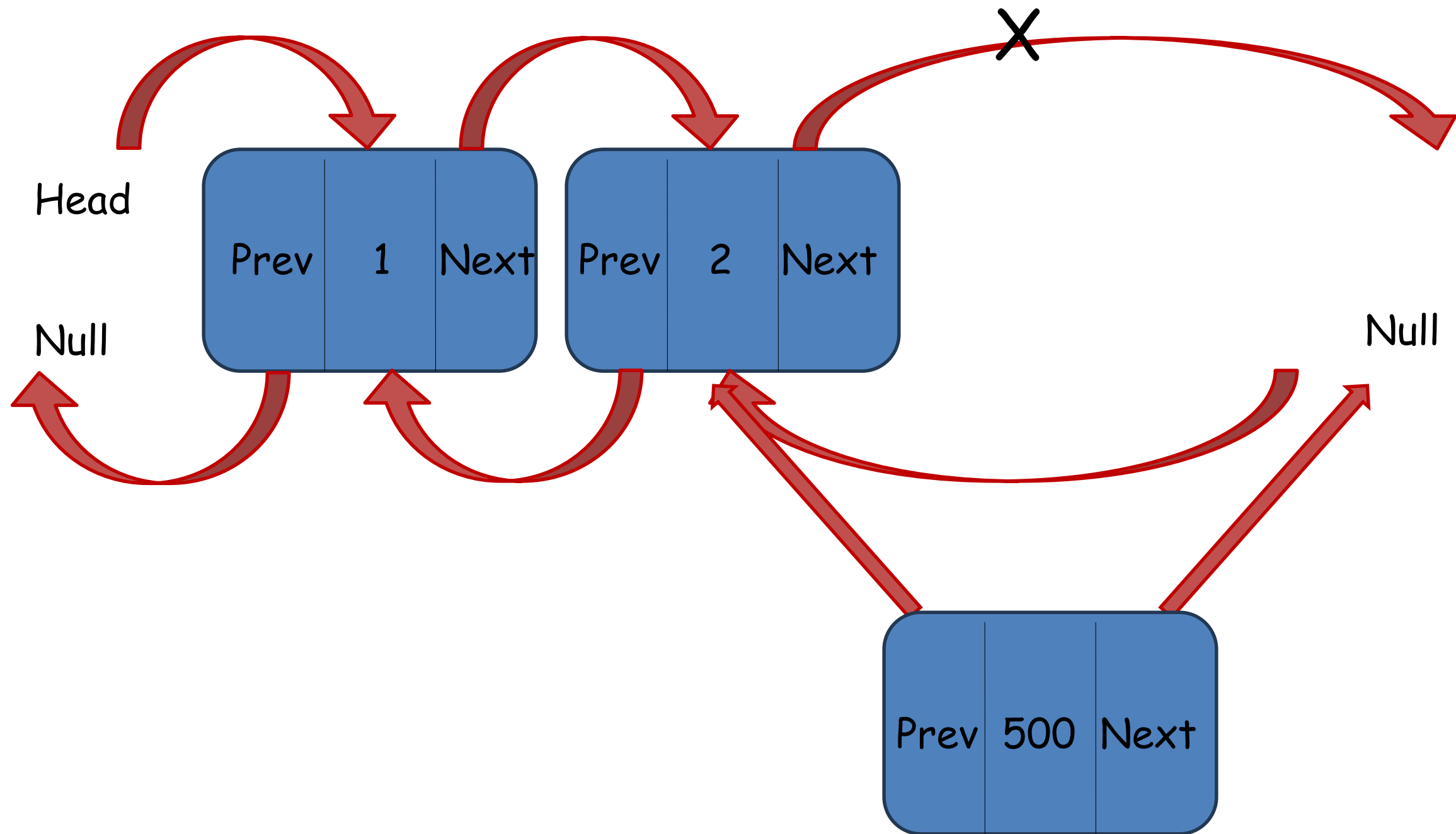


Data Structure

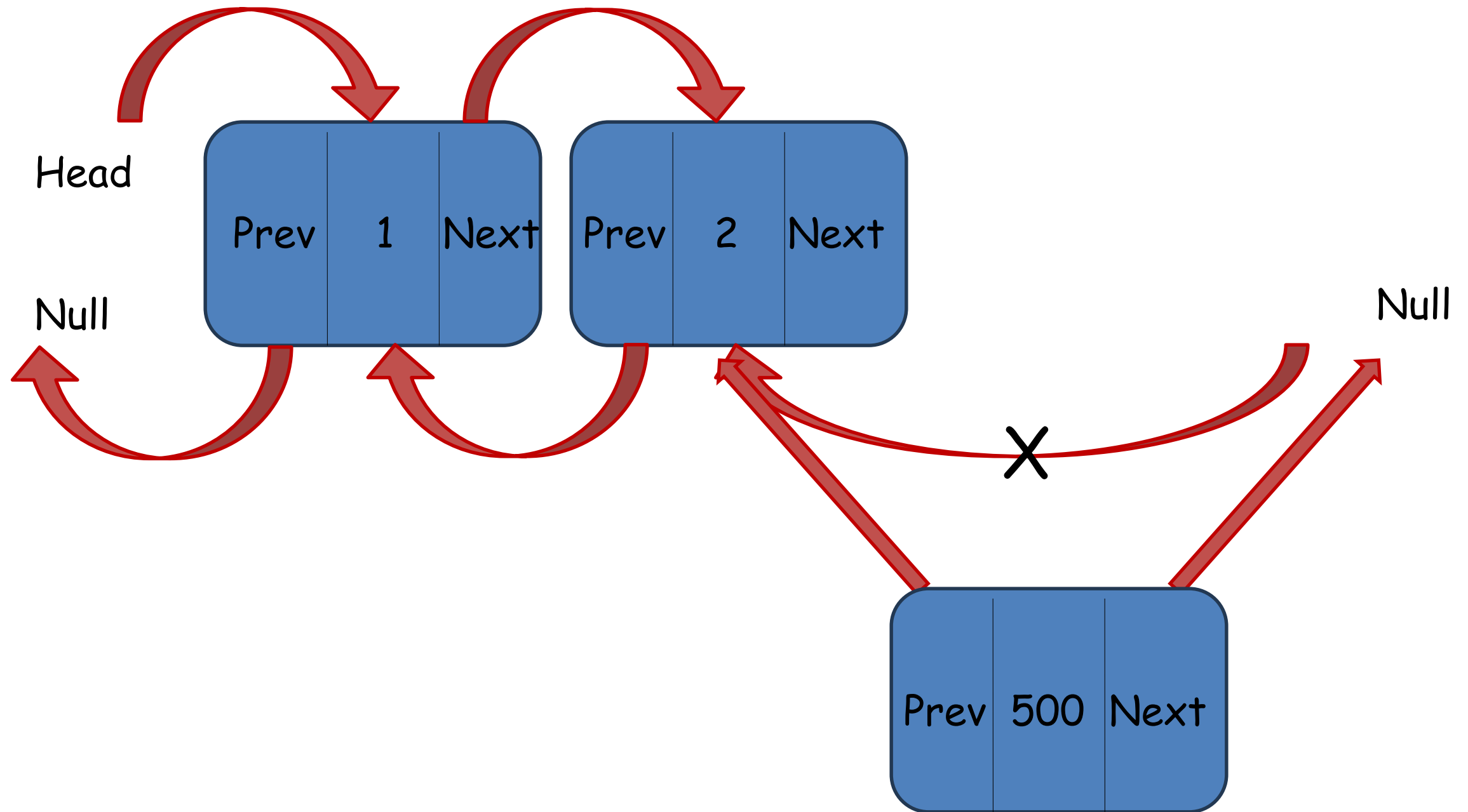
Level 1

Insert At End

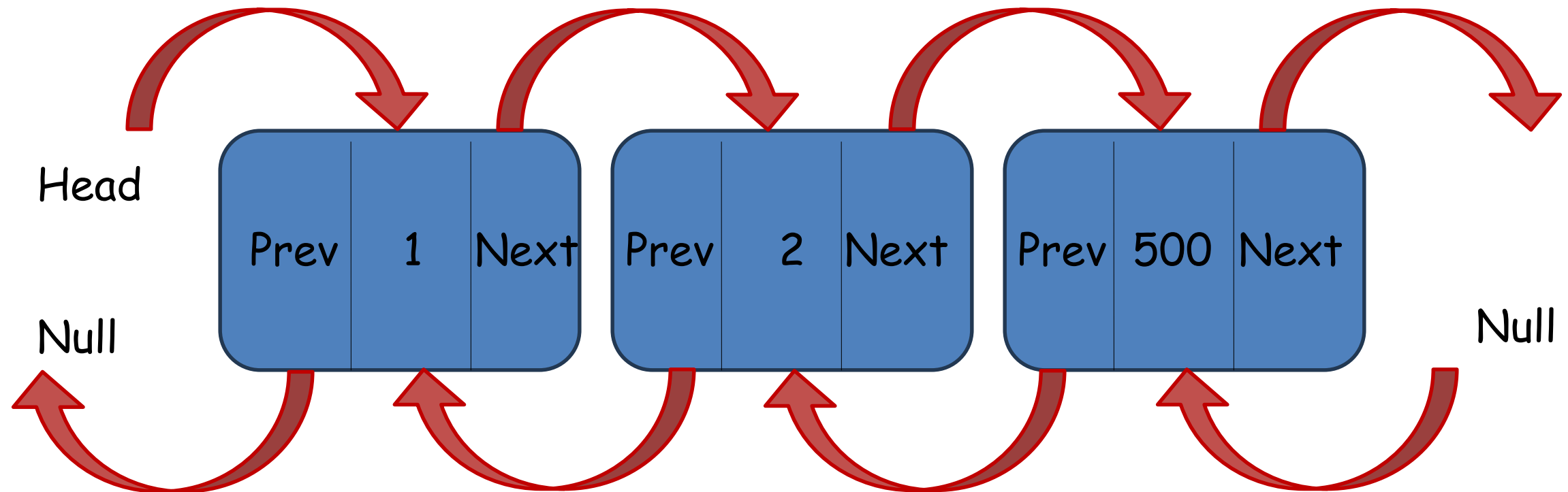
Doubley Linked List Data Structure



Doubley Linked List Data Structure



Doubley Linked List Data Structure



Definition

Adds a new node at the end (tail) of the doubly linked list.

Explanation

- ◆ This operation inserts a node at the last position in the list
- ◆ It requires traversing the list to reach the final node

□ Procedures

1 Create New Node

□ Steps:

- Allocate new node
- Set value
- Set next = NULL

□ Procedures

2 □ Check if List is Empty

- If head == NULL:

- Set newNode->prev = NULL

- Make head = newNode

→ □ Result: First node in the list

□ Procedures

3 □ Traverse to Last Node

□ Steps:

- Start from head
- Move until `current->next == NULL`

□ Procedures

4 Insert at End

□ Steps:

□ Set `current->next = newNode`

□ Set `newNode->prev = current`

→ □ Result: Node added at the end

💡 Example

Before:

NULL $\leftrightarrow$ 1 $\leftrightarrow$ 2 $\leftrightarrow$ 3 $\leftrightarrow$ 4 $\leftrightarrow$ 5 $\rightarrow$ NULL

After inserting 500:

NULL $\leftrightarrow$ 1 $\leftrightarrow$ 2 $\leftrightarrow$ 3 $\leftrightarrow$ 4 $\leftrightarrow$ 5 $\leftrightarrow$ 500 $\rightarrow$ NULL

Note

Traversal is required unless you keep a tail pointer
Works for both empty and non-empty lists

⊘ Warning

⚠ In this code, head is passed by value, so if the list is empty, the change will NOT affect the original head

⚖️ Comparison

□ Insert At Beginning

- ✓□ Time Complexity: $O(1)$

- ✓□ No traversal needed

- ✓□ Direct head update

□ Insert At End

- ✓□ Time Complexity: $O(n)$

- ✓□ Requires traversal

- ✓□ Updates last node pointers

✦✦ Summary

- ✓ InsertAtEnd adds a node at the last position
- ✓ Requires traversal to reach the last node
- ✓ Updates both next and prev pointers
- ✓ Can be optimized using a tail pointer

Thank you 😊